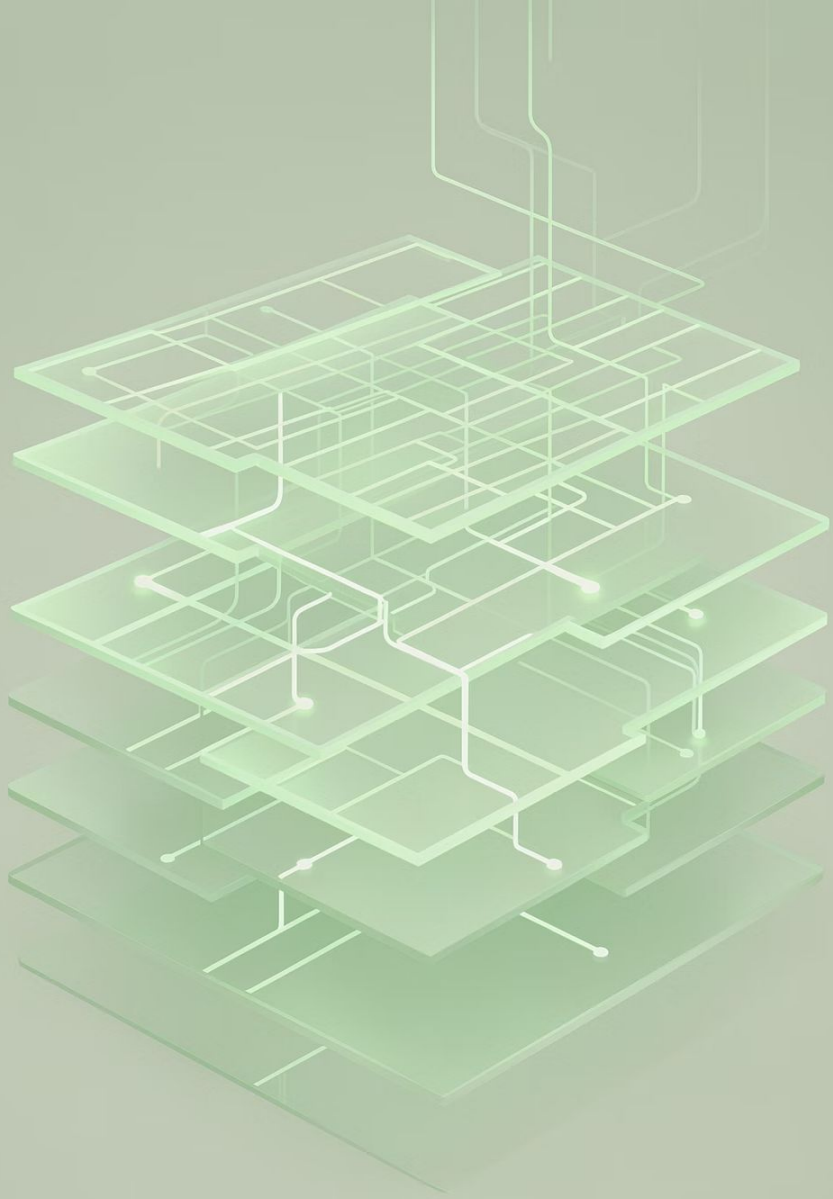




Redes Convolucionales (CNN)

Sesión 11.1 · Especialista en IA — Visión por Computador

MÓDULO 11

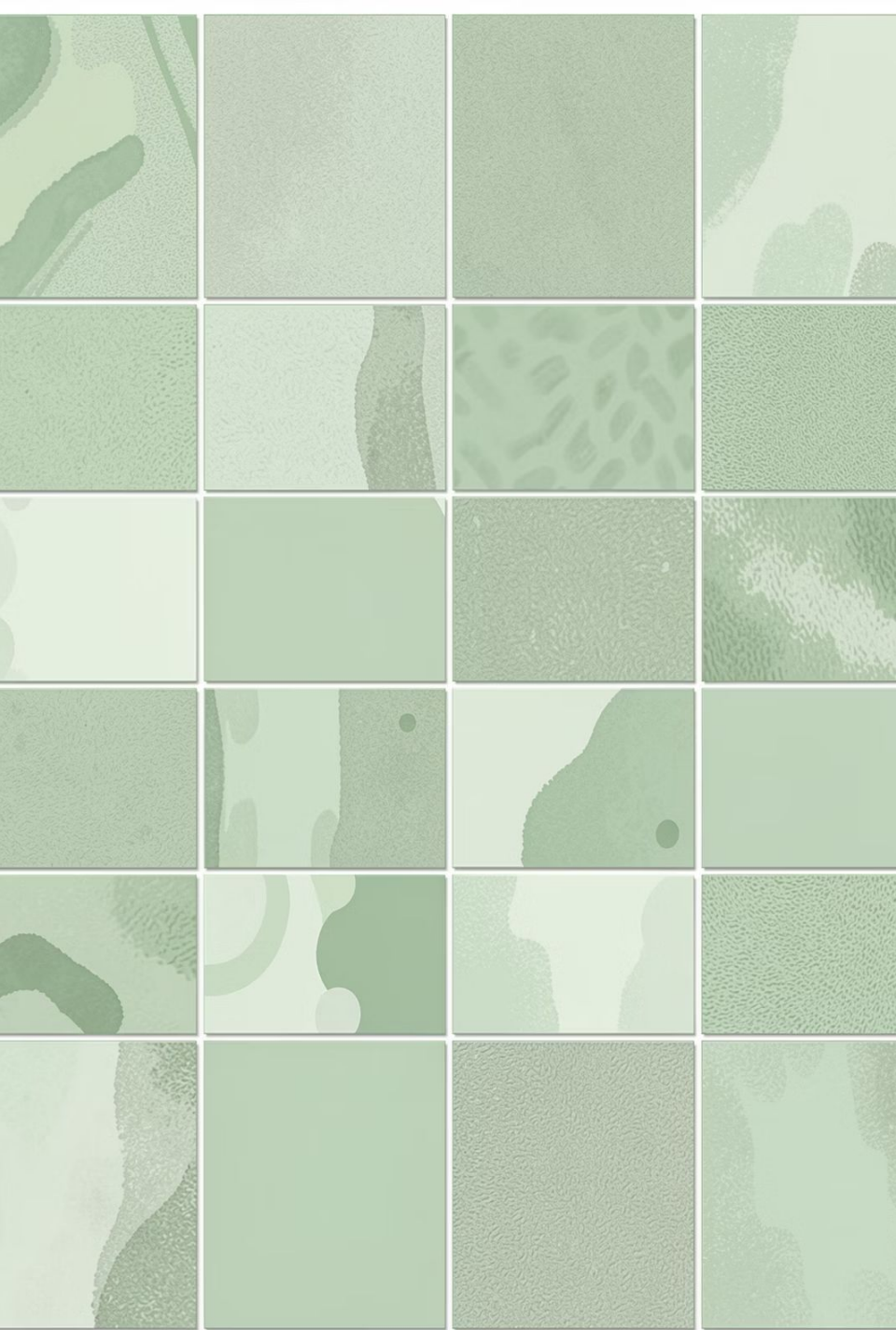
¿Por qué no basta un MLP para imágenes?



El problema de las imágenes planas

Un MLP recibe un vector 1D: si aplanamos una imagen de $224 \times 224 \times 3$, obtenemos **150.528 valores**. Cada neurona de la primera capa necesita conectarse a todos ellos → millones de parámetros, sobreajuste garantizado.

⚠ Aplanar destruye la información espacial: el píxel vecino ya no "sabe" que es vecino.



Localidad y equivarianza al desplazamiento

Localidad

Los patrones visuales (bordes, texturas) aparecen en regiones pequeñas, no en toda la imagen.

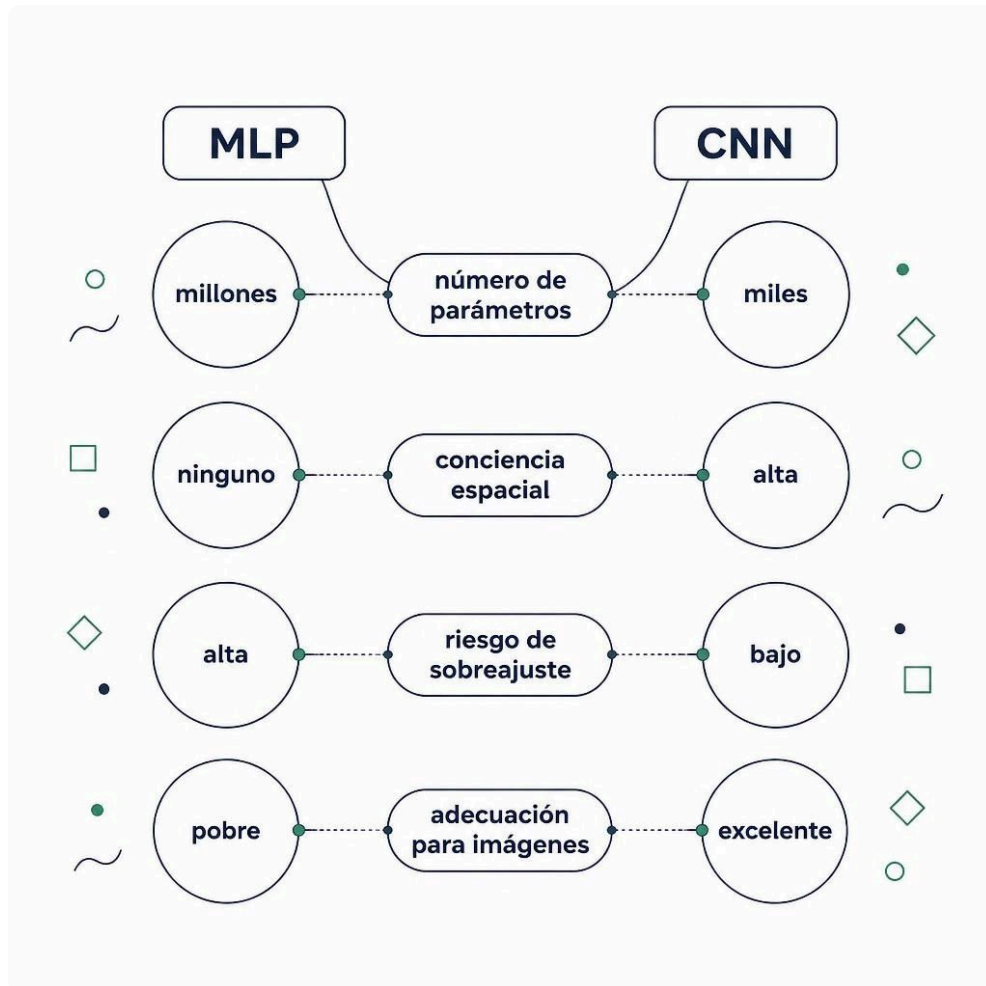
Compartición de pesos

El mismo filtro se aplica en toda la imagen, reduciendo drásticamente los parámetros.

Equivarianza

Si el objeto se desplaza, la respuesta del filtro también se desplaza: la red lo detecta en cualquier posición.

MLP vs CNN: una comparación directa



¿Qué gana una CNN?

Las CNN introducen dos restricciones que resultan ser ventajas: **conectividad local** (cada neurona ve solo una pequeña región) y **pesos compartidos** (un único filtro barre toda la imagen). El resultado es una red mucho más eficiente y robusta.

APARTADO 2

La Convolución

La operación central de toda CNN: deslizar un pequeño filtro (kernel) sobre la imagen y calcular el producto escalar en cada posición.

$$\begin{matrix} 4 & 5 & 6 & 7 & 8 & 9 & 10 & 11 & 12 \\ 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 \\ 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 \\ 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 & 11 \\ 4 & 5 & 6 & 7 & 8 & 9 & 10 & 11 & 12 \\ 5 & 6 & 7 & 8 & 9 & 10 & 11 & 12 & 13 \\ 6 & 7 & 8 & 9 & 10 & 11 & 12 & 13 & 14 \\ 7 & 8 & 9 & 10 & 11 & 12 & 13 & 14 & 15 \\ 8 & 9 & 10 & 11 & 12 & 13 & 14 & 15 & 16 \end{matrix}$$

	8-6		4	-	5	
4	3-7	6	4	9	10	4
1	6-6	9	7	5	4	5
1	3-6	6	0	4	9	2
3	3-7	9	8	2	7	3
4	5 5	5	10	9	4	1%
3	3-4	4	19	3	7	71
6	6-5	13	7	2	7	6
9	3 1	3	3	4	4	3
2	8-7	2	%	5	4	9

¿Cómo funciona matemáticamente?

La fórmula

$$(I * K)[i, j] = \sum_m \sum_n I[i + m, j + n] \cdot K[m, n]$$

Donde **I** es la imagen de entrada y **K** es el kernel. El resultado es el **mapa de características** (feature map).

En la práctica, los frameworks de deep learning optimizan esta operación con GPU de forma muy eficiente.



Colocar Kernel

Multiplicar Elementos

Sumar Resultado

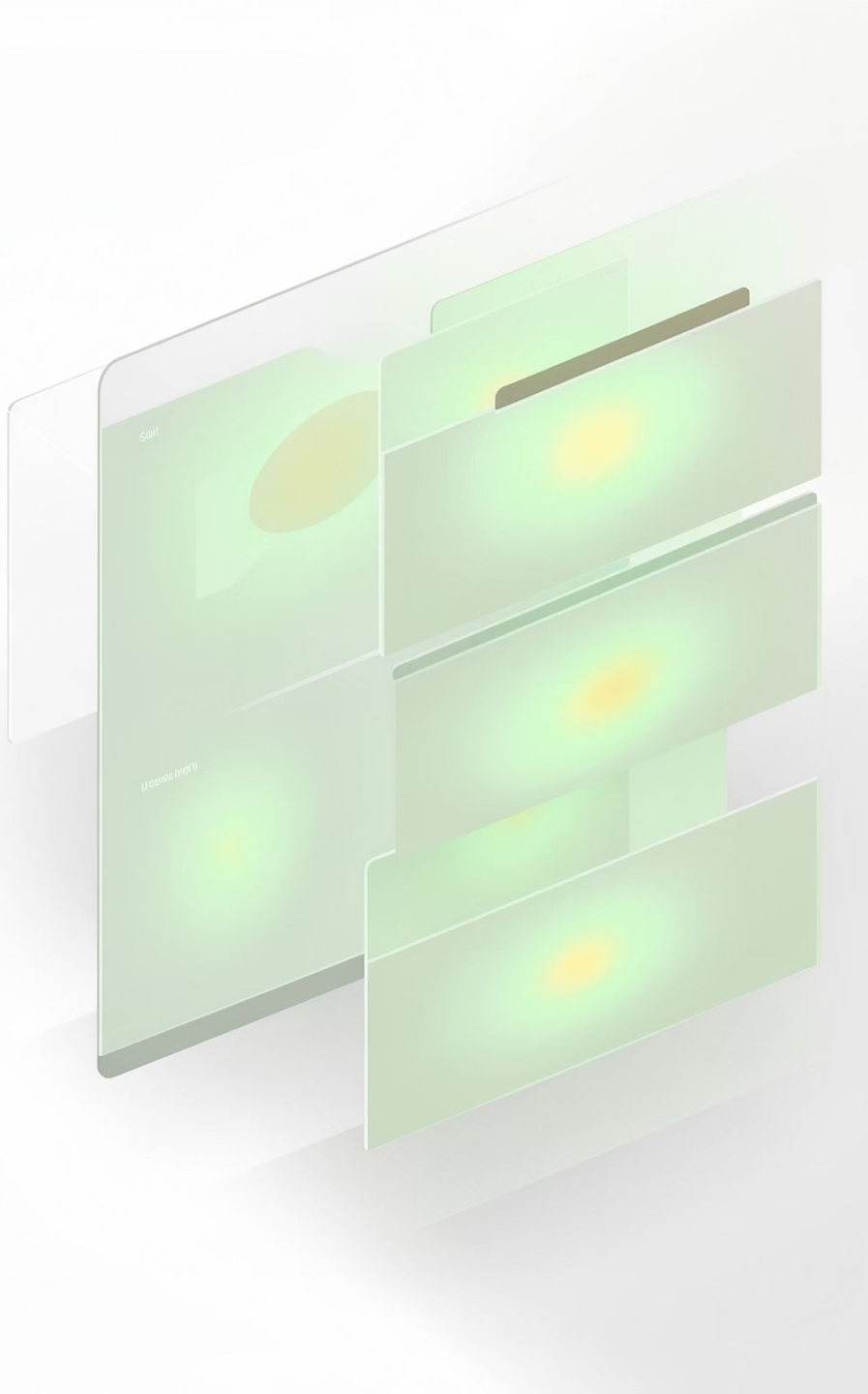
Detectando bordes en una foto de alojamiento



Ejemplo TurisData Canarias

Imaginemos que queremos clasificar fotos de alojamientos turísticos en Gran Canaria. Un filtro de bordes **Sobel** detecta automáticamente contornos de muebles, ventanas y paredes. Esto es exactamente lo que aprende la primera capa de una CNN: bordes, esquinas y texturas básicas.

- ❏ No programamos el filtro — la red lo aprende sola durante el entrenamiento.



APARTADO 3

Filtros y Mapas de Características

Cada filtro aprende a detectar un patrón distinto. La colección de todos los mapas de activación forma una **representación jerárquica** de la imagen.

De píxeles a semántica: la jerarquía de filtros



Capa 1

Bordes y gradientes de color



Capa 2

Texturas y esquinas



Capa 3

Formas y partes de objetos



Capa final

Objetos completos y conceptos

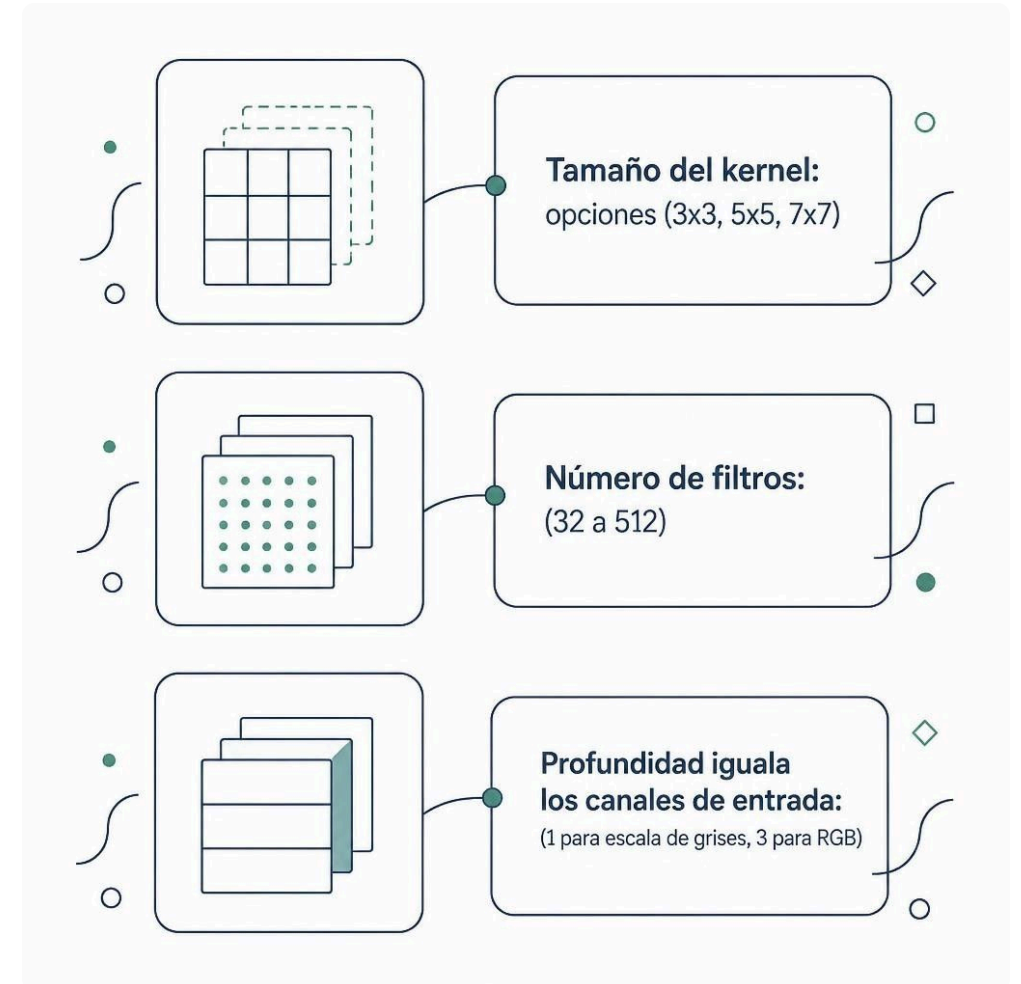
Una CNN con 64 filtros por capa genera 64 mapas de características distintos — cada uno respondiendo a un patrón diferente en la imagen.

¿Cuántos filtros y de qué tamaño?

Hiperparámetros clave

- **Tamaño del kernel:** 3×3 (lo más habitual), 5×5, 7×7
- **Número de filtros:** 32, 64, 128... va aumentando en capas profundas
- **Profundidad de entrada:** para RGB son 3 canales, el filtro también tiene profundidad 3

❏ Kernels 3×3 apilados son tan expresivos como un 7×7 y usan menos parámetros.



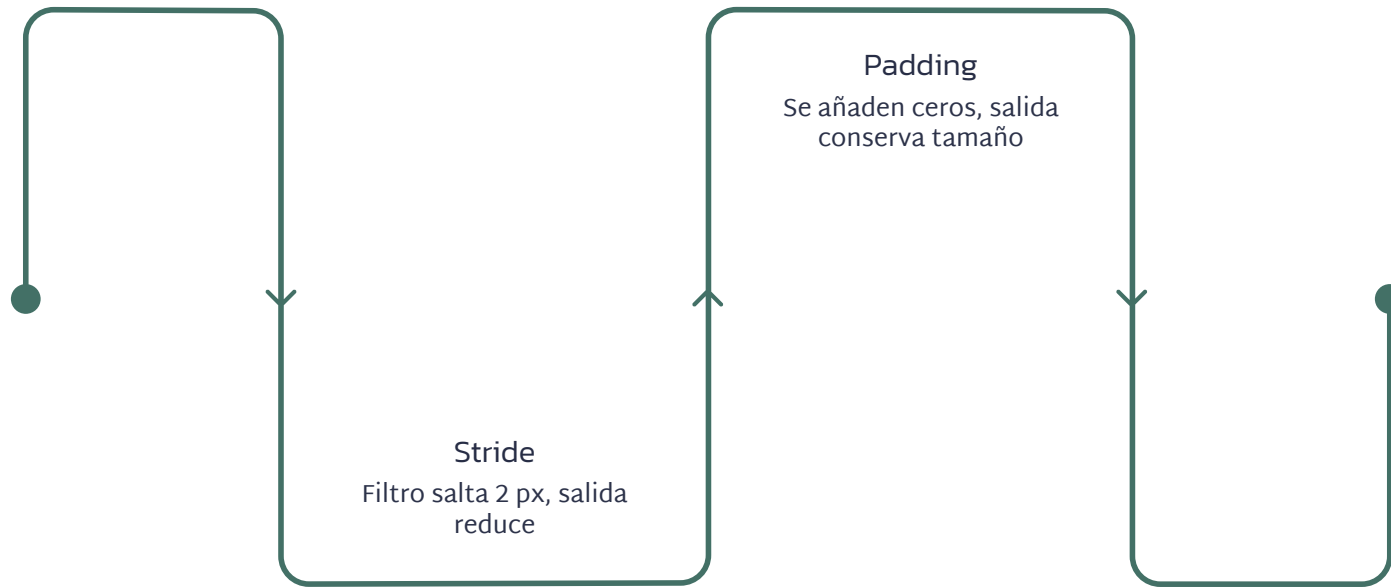
APARTADO 4

Pooling, Padding y Stride

Tres mecanismos que controlan cómo fluye y se reduce la información a través de la red.



Stride y Padding



Stride

Cuánto se desplaza el filtro en cada paso. Stride=2 reduce el tamaño del mapa a la mitad y acelera el cómputo.

Padding

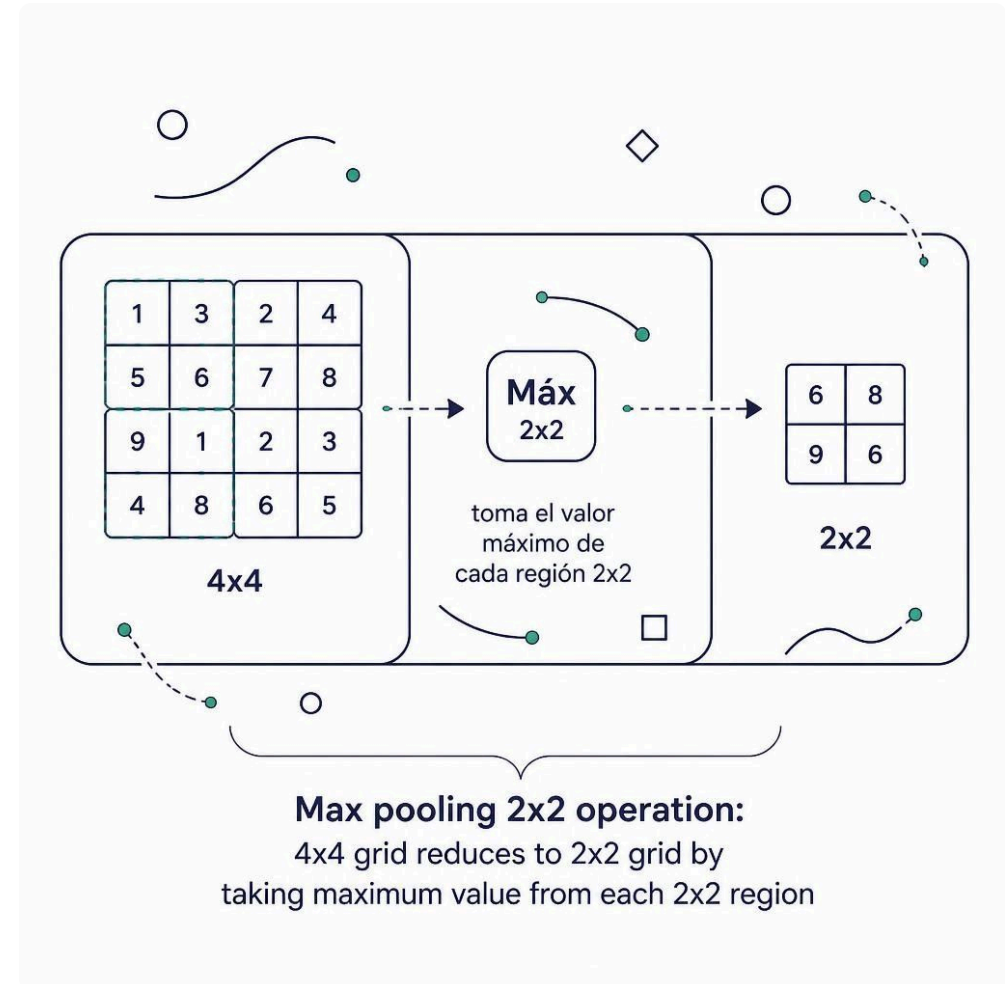
Añadir ceros en el borde (*zero-padding*) para que la salida mantenga las mismas dimensiones que la entrada (**same padding**) o las reduzca (**valid padding**).

Max Pooling: comprimiendo sin perder lo importante

¿Qué hace?

Divide el mapa de características en regiones pequeñas (típicamente 2x2) y se queda con el valor máximo de cada una. Reduce el tamaño a la mitad y aporta **invarianza a pequeñas traslaciones**.

- **Max Pooling:** valor máximo (el más habitual)
- **Average Pooling:** media de la región
- **Global Average Pooling:** media de todo el mapa — se usa al final de arquitecturas modernas



Arquitectura Típica de una CNN

Bloque extractor de características + bloque clasificador. Esta estructura se repite en casi todas las CNN modernas.



El bloque convolucional en detalle



Convolución

El filtro desliza sobre la entrada y genera un mapa de activación.



Activación ReLU

Se aplica ReLU para introducir no-linealidad: $\max(0, x)$.



Pooling

Se reduce el tamaño espacial manteniendo las características más relevantes.



Repetir N veces

Apilar varios bloques aumenta la profundidad semántica de la representación.

Clasificar fotos de alojamientos en Canarias

Caso TurisData Canarias

Una CNN puede clasificar automáticamente imágenes de portales turísticos en categorías como **piscina, vista al mar, interior** o **exterior**. El bloque convolucional extrae texturas y formas; las capas densas finales deciden la categoría.

- ❏ Este tipo de clasificación automatiza tareas de catalogación que llevarían horas a un equipo humano.



Arquitecturas Clásicas

LeNet-5 · 1998

Primera CNN exitosa. Reconocimiento de dígitos escritos a mano (MNIST).

VGG-16 · 2014

Profundidad con kernels 3×3 . Simple y muy reutilizada.

1

2

3

4

AlexNet · 2012

Ganó ImageNet con GPU. Popularizó ReLU y Dropout.

ResNet · 2015

Conexiones residuales. Permite redes de 50–152 capas sin degradación.

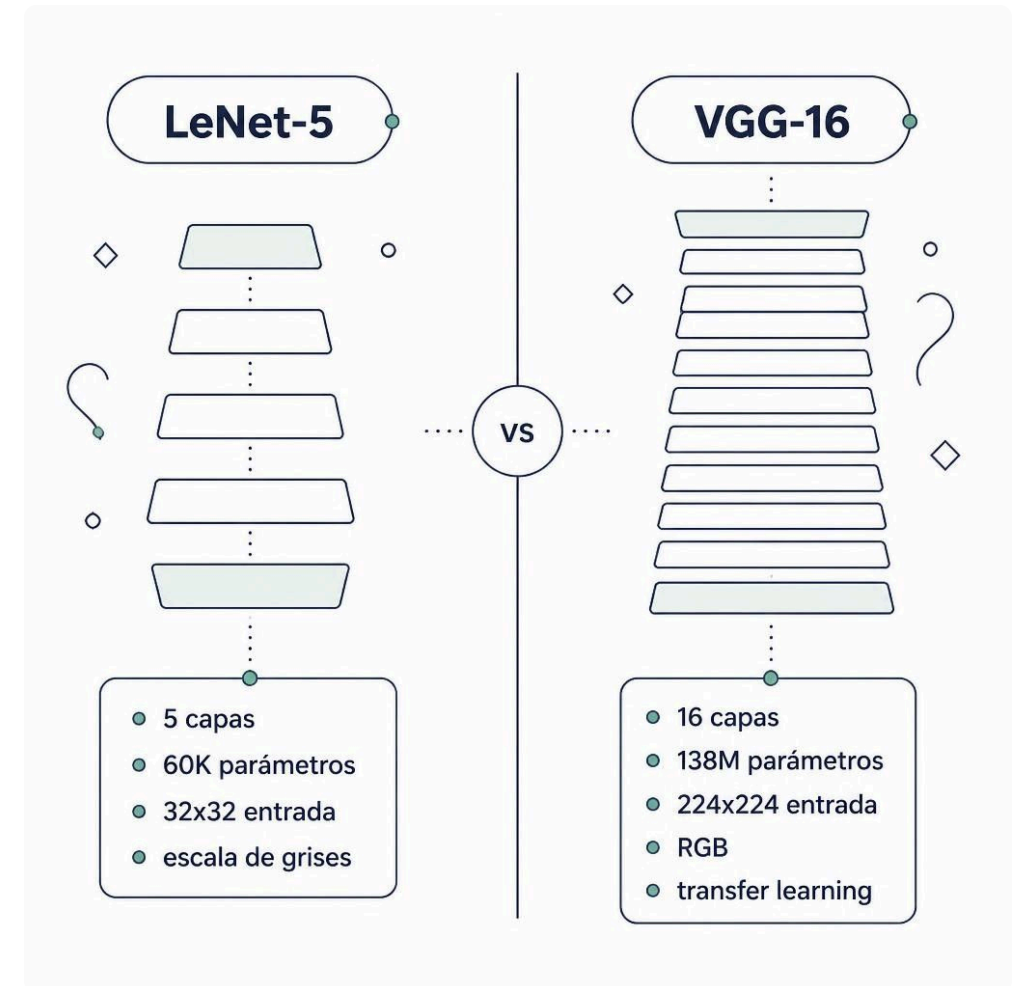
LeNet y VGG: profundidad con simplicidad

LeNet-5

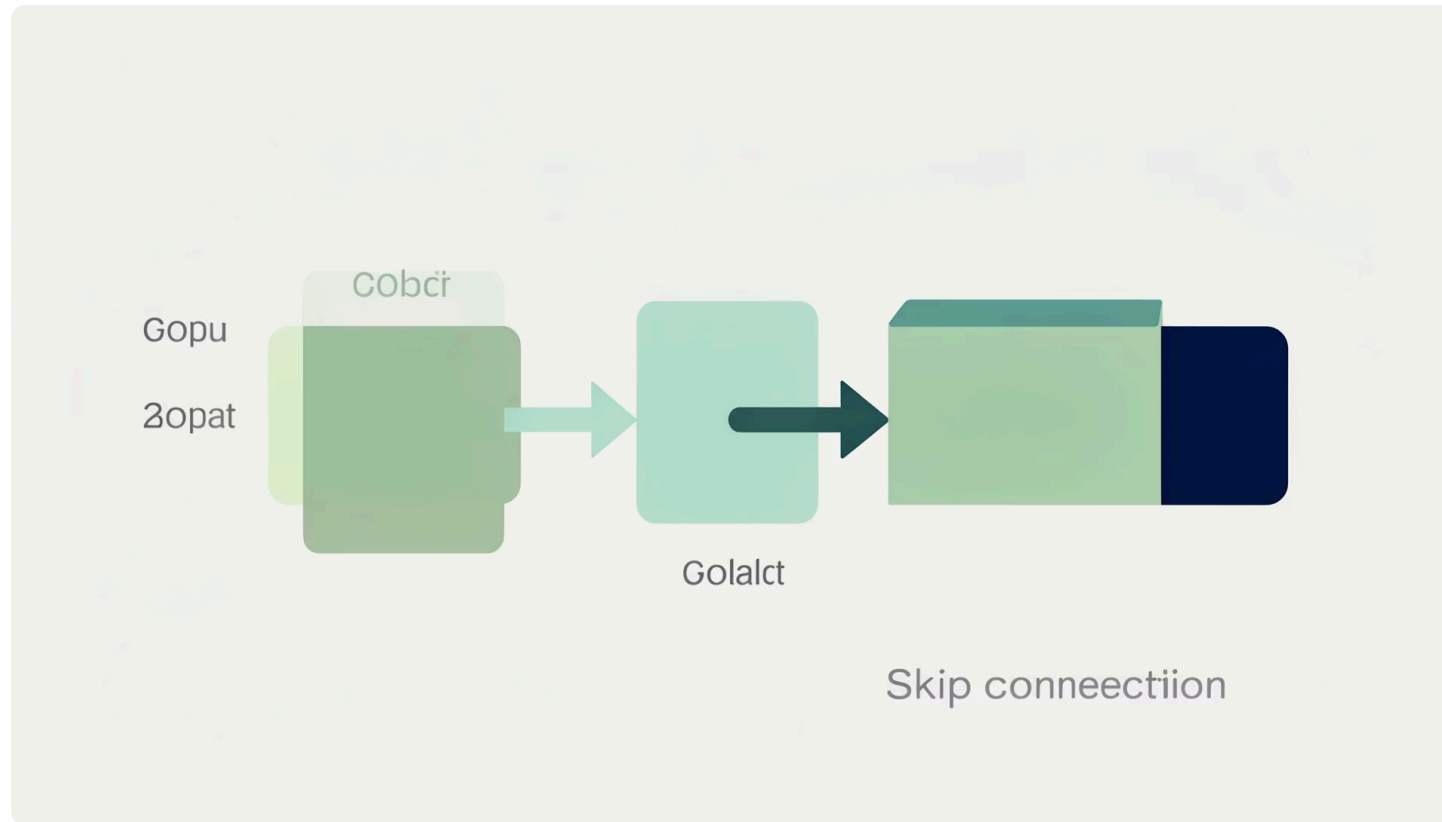
Solo 5 capas entrenables, ~60 000 parámetros. Diseñada para imágenes en escala de grises 32×32. Sentó las bases de la arquitectura Conv → Pool → FC.

VGG-16

16 capas entrenables, ~138 M de parámetros. Bloques homogéneos de convoluciones 3×3. Muy usada como **extractor de características** en transfer learning.



ResNet: el salto que lo cambió todo



El problema del gradiente que desaparece

Al apilar muchas capas, el gradiente se vuelve tan pequeño que las capas inferiores dejan de aprender. ResNet introduce **conexiones residuales**: la señal original "salta" el bloque y se suma a la salida.

$$H(x) = F(x) + x$$

Esta simple adición permite entrenar redes de más de 100 capas con total estabilidad.

APARTADO 7

Data Augmentation

Generar versiones modificadas de las imágenes de entrenamiento para que la red aprenda a ser robusta ante variaciones del mundo real.



Técnicas de aumentación de datos



Volteo y rotación

Flip horizontal/vertical, rotaciones aleatorias ($\pm 15^\circ$). Básico y muy efectivo.



Recorte y zoom

Random crop y zoom simulan diferentes encuadres de cámara.



Color jitter

Cambios de brillo, contraste y saturación — clave para fotos de alojamientos con distintas condiciones de luz.



Cutout / Mixup

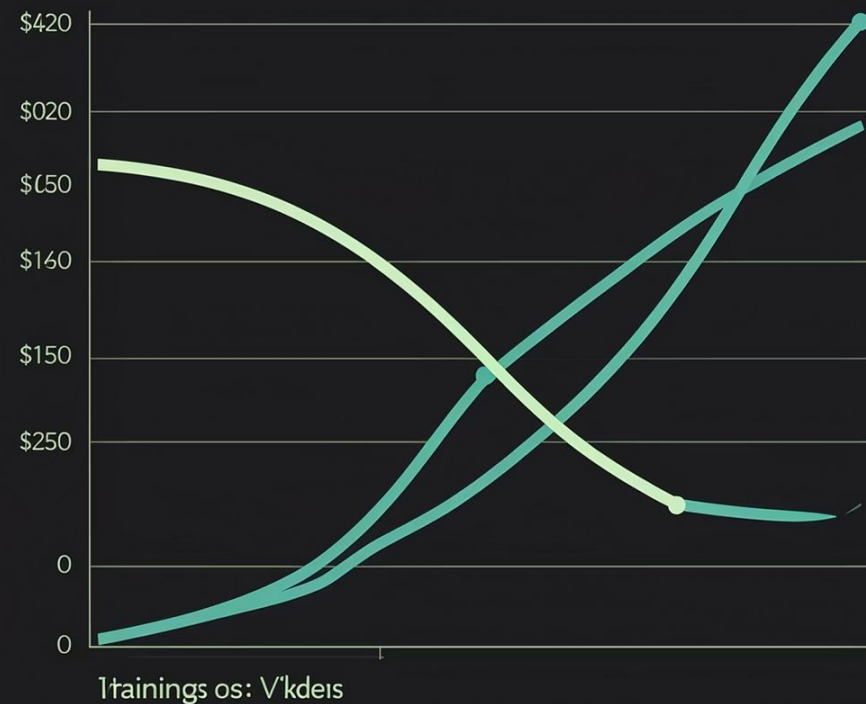
Técnicas avanzadas: borrar regiones aleatorias o mezclar dos imágenes para mejorar la generalización.

APARTADO 8

Entrenamiento y Buenas Prácticas

Entrenar una CNN correctamente requiere más que datos y GPU: hay decisiones de diseño que marcan la diferencia entre un modelo que generaliza y uno que sobreajusta.

Training Loss & Validation Loss Curve



Receta de entrenamiento efectivo

Transfer Learning primero

Parte de ResNet o VGG preentrenados en ImageNet. Congela las capas base y entrena solo el clasificador final. Ahorra tiempo y datos.

Learning Rate Scheduling

Empieza con un LR bajo, usa *warmup* y reduce con *ReduceLROnPlateau* o *cosine annealing*.

Batch Normalization y Dropout

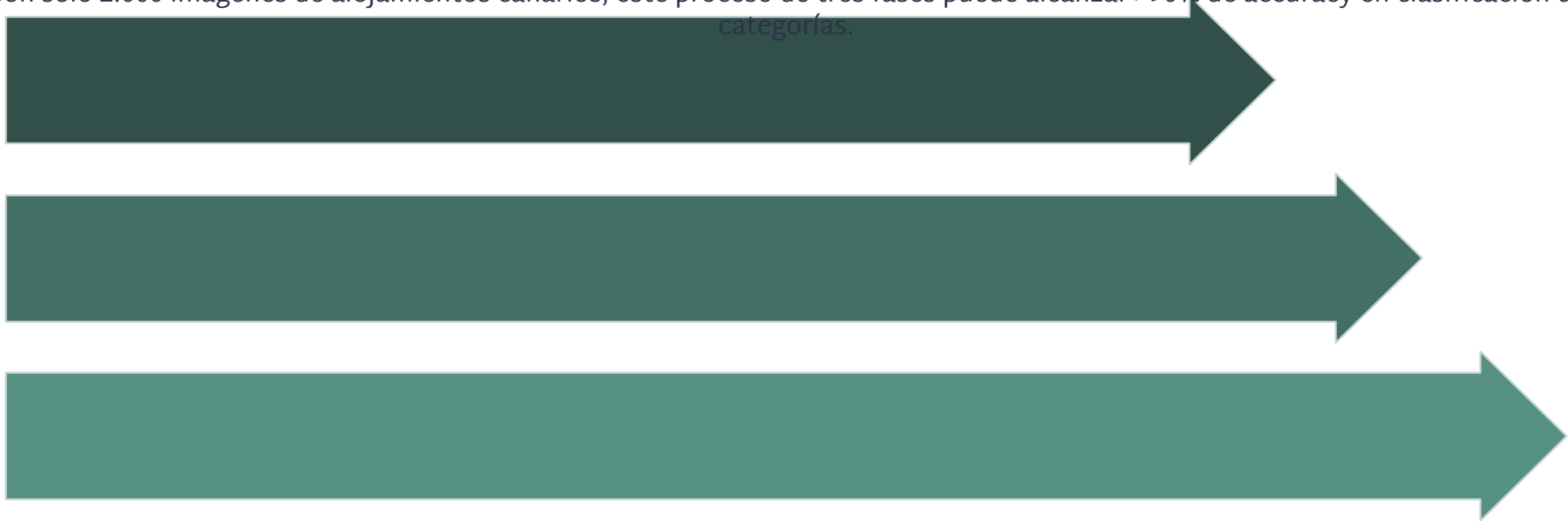
BatchNorm estabiliza el entrenamiento. Dropout (0.3–0.5) en las capas densas reduce el sobreajuste.

Early Stopping

Detén el entrenamiento cuando la pérdida de validación deje de mejorar. Guarda el mejor checkpoint.

Fine-tuning para fotos de alojamientos

Con solo 2.000 imágenes de alojamientos canarios, este proceso de tres fases puede alcanzar >90% de accuracy en clasificación de categorías.



Interpretar lo que Aprende la Red



¿Es una caja negra o podemos mirar dentro?

Las CNN no tienen por qué ser opacas. Existen varias técnicas para visualizar qué partes de la imagen activan cada neurona y qué patrones han aprendido los filtros.

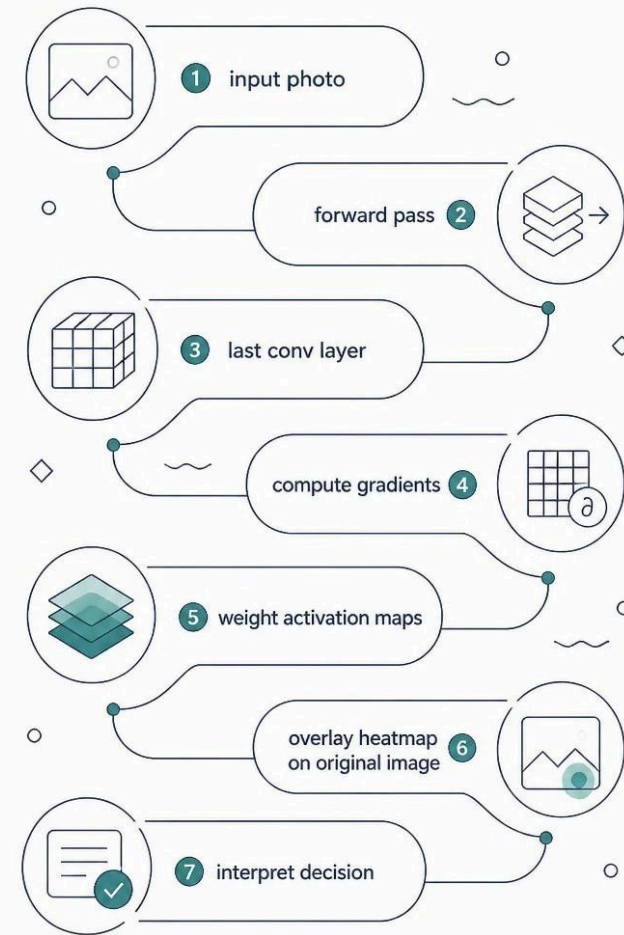
- **Visualización de filtros:** ver los kernels aprendidos
- **Mapas de activación:** ver qué regiones responden a qué
- **Grad-CAM:** mapa de calor sobre la imagen original

Grad-CAM: "¿Dónde mira la red?"

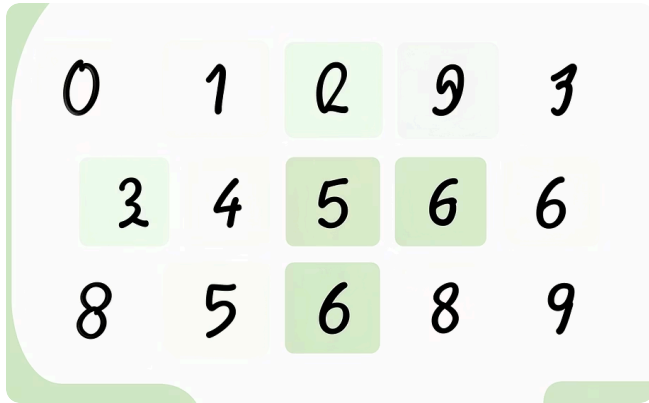
Cómo funciona

Grad-CAM calcula el gradiente de la clase predicha respecto a los mapas de la última capa convolucional. Las regiones con gradiente alto son las que más influyen en la decisión.

Aplicación en TurisData: si la red predice "piscina", Grad-CAM debería resaltar el área de agua — si resalta la pared, algo va mal en los datos.



Ejemplo Práctico: MNIST y CIFAR-10



MNIST

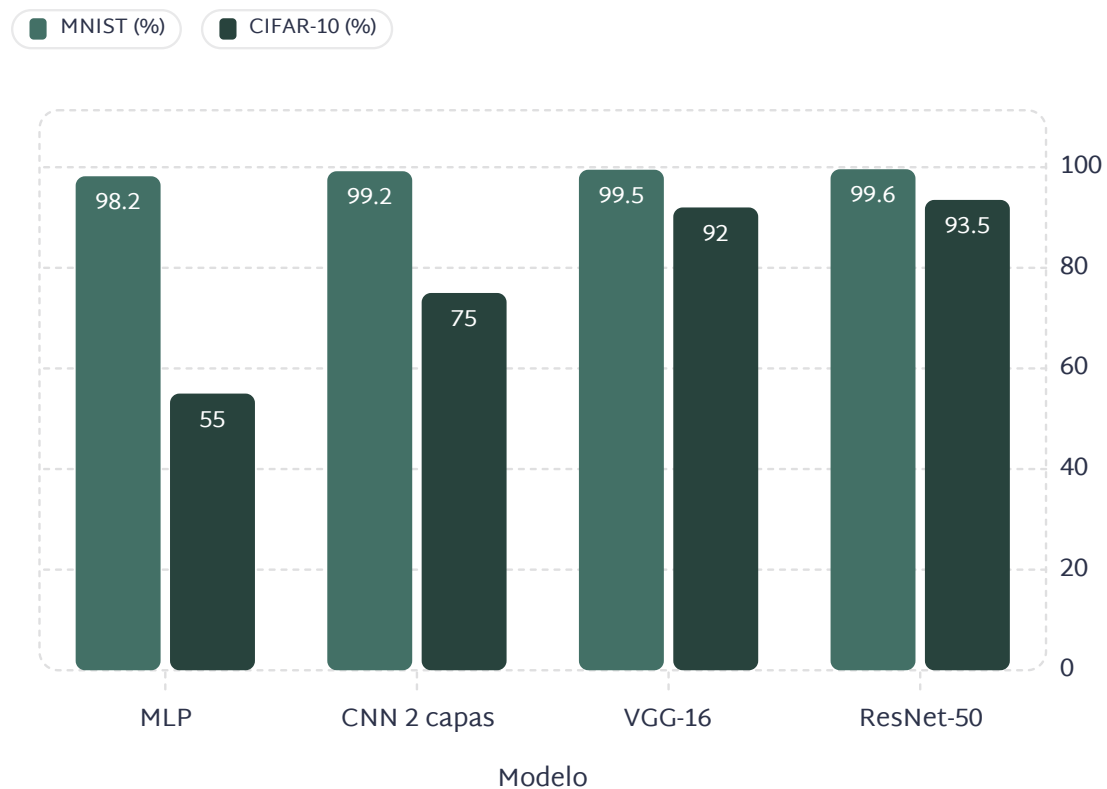
60 000 dígitos escritos a mano, 28×28 px, escala de grises.
Una CNN de 2 capas supera el 99% de accuracy. El punto de partida clásico.



CIFAR-10

60 000 imágenes 32×32 RGB en 10 clases. Mucho más difícil:
necesita arquitecturas más profundas y data augmentation para superar el 90%.

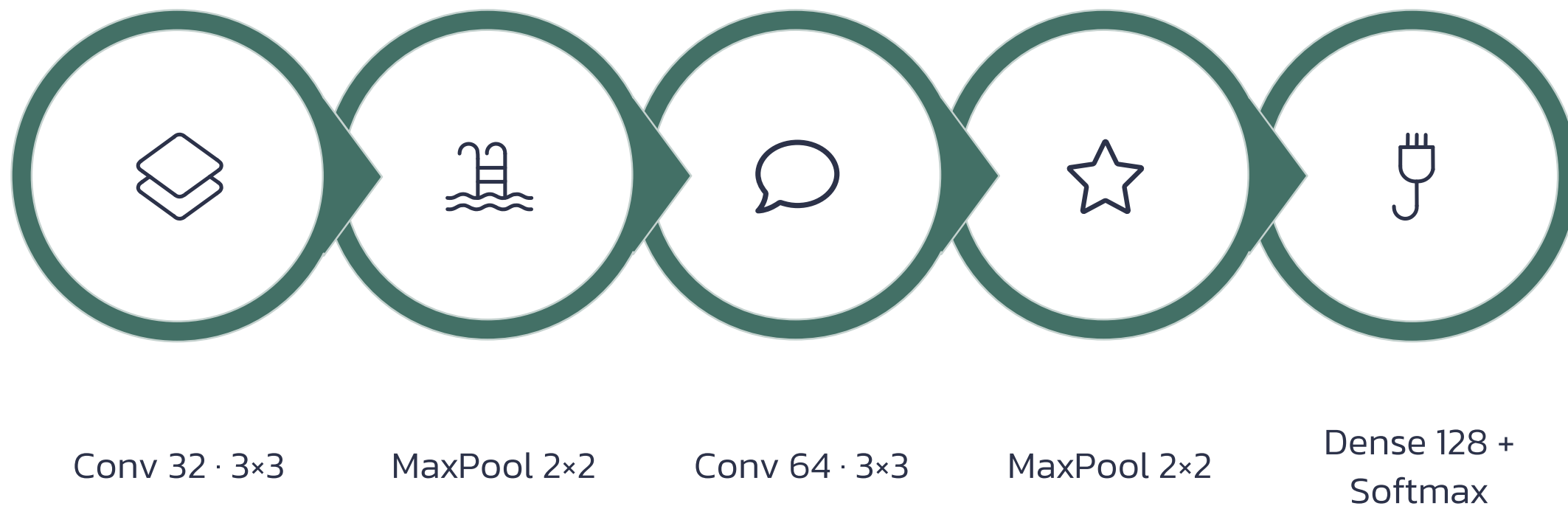
Resultados reales con CNN sencillas



Lo que nos dicen los datos

En MNIST, incluso un MLP rinde bien — la tarea es relativamente simple. En CIFAR-10, la brecha entre MLP y CNN se dispara: la información espacial importa mucho. Las arquitecturas más profundas (VGG, ResNet) siguen mejorando gracias a su mayor capacidad de abstracción.

Esquema de las 5 Capas Entrenables



Esta arquitectura de 5 capas entrenables, implementada en Keras en menos de 20 líneas, puede superar el 80% de accuracy en CIFAR-10 en minutos de entrenamiento.

Puntos Clave de la Sesión

Motivación

Los MLP destruyen la estructura espacial; las CNN la preservan con localidad y pesos compartidos.

Mecanismo

Convolución + ReLU + Pooling forman el bloque básico. Los filtros aprenden jerarquías de representación.

Arquitecturas

LeNet → VGG → ResNet: más profundidad, mejor generalización. Usa transfer learning siempre que puedas.

En la práctica

Data augmentation, fine-tuning y Grad-CAM son tus aliados para entrenar, adaptar e interpretar una CNN real.

